

PRACTICAL 32 -

Objective - **Minimum Spanning Tree using Kruskal's or Prim's Algorithm**

Construct a minimum spanning tree of a given weighted undirected graph.

CODE -

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <stdbool.h>
4
5  #define MAX_NODES 100
6  #define INF 999999
7
8  // Structure to represent an edge in the graph
9  typedef struct Edge {
10     int src, dest, weight;
11 } Edge;
12
13 // Structure to represent a graph
14 typedef struct Graph {
15     int numVertices, numEdges;
16     Edge edges[MAX_NODES];
17 } Graph;
18
19 // Structure to represent a subset for Union-Find
20 typedef struct Subset {
21     int parent;
22     int rank;
23 } Subset;
24
25 // Function to create a new graph
26 Graph* create_graph(int numVertices, int numEdges) {
27     Graph* graph = (Graph*)malloc(sizeof(Graph));
28     graph->numVertices = numVertices;
29     graph->numEdges = numEdges;
30     return graph;
31 }
32
33 // Comparator function to sort edges by weight
34 int compare_edges(const void* a, const void* b) {
35     Edge* edgeA = (Edge*)a;
36     Edge* edgeB = (Edge*)b;
37     return edgeA->weight - edgeB->weight;
38 }
39
40 // Find function for Union-Find
41 int find(Subset subsets[], int i) {
```

```

42     if (subsets[i].parent != i) {
43         subsets[i].parent = find(subsets, subsets[i].parent);
44     }
45     return subsets[i].parent;
46 }
47
48 // Union function for Union-Find
49 void union_subsets(Subset subsets[], int x, int y) {
50     int rootX = find(subsets, x);
51     int rootY = find(subsets, y);
52
53     if (subsets[rootX].rank < subsets[rootY].rank) {
54         subsets[rootX].parent = rootY;
55     } else if (subsets[rootX].rank > subsets[rootY].rank) {
56         subsets[rootY].parent = rootX;
57     } else {
58         subsets[rootY].parent = rootX;
59         subsets[rootX].rank++;
60     }
61 }
62
63 // Kruskal's Algorithm to find Minimum Spanning Tree
64 void kruskal_mst(Graph* graph) {
65     int numVertices = graph->numVertices;
66     Edge result[MAX_NODES]; // Array to store the resultant MST
67     int resultIndex = 0;    // Index for result array
68
69     // Step 1: Sort all edges in non-decreasing order of their weight
70     qsort(graph->edges, graph->numEdges, sizeof(graph->edges[0]),
71         compare_edges);
72
73     // Allocate memory for Union-Find subsets
74     Subset* subsets = (Subset*)malloc(numVertices * sizeof(Subset));
75     for (int v = 0; v < numVertices; v++) {
76         subsets[v].parent = v;
77         subsets[v].rank = 0;
78     }
79
80     // Step 2: Pick the smallest edge and check if it forms a cycle
81     for (int i = 0; i < graph->numEdges && resultIndex < numVertices - 1; i++) {
82         Edge nextEdge = graph->edges[i];
83         int x = find(subsets, nextEdge.src);
84         int y = find(subsets, nextEdge.dest);
85
86         // If including this edge does not form a cycle
87         if (x != y) {
88             result[resultIndex++] = nextEdge;
89             union_subsets(subsets, x, y);
90         }
91     }
92 }

```

```

91
92     // Print the resultant MST
93     printf("Edges in the Minimum Spanning Tree (Kruskal's Algorithm):\n
94         ");
95     for (int i = 0; i < resultIndex; i++) {
96         printf("%d -- %d == %d\n", result[i].src, result[i].dest,
97             result[i].weight);
98     }
99     free(subsets);
100 }
101 // Prim's Algorithm to find Minimum Spanning Tree
102 void prim_mst(int graph[MAX_NODES][MAX_NODES], int numVertices) {
103     int parent[MAX_NODES]; // Array to store constructed MST
104     int key[MAX_NODES];    // Key values used to pick minimum weight
105                             edge
106     bool mstSet[MAX_NODES]; // To represent vertices included in MST
107
108     // Initialize all keys as INFINITE
109     for (int i = 0; i < numVertices; i++) {
110         key[i] = INF;
111         mstSet[i] = false;
112     }
113
114     // Always include the first vertex in MST
115     key[0] = 0;
116     parent[0] = -1; // First node is always the root of MST
117
118     // MST construction
119     for (int count = 0; count < numVertices - 1; count++) {
120         // Find the minimum key vertex not yet included in MST
121         int min = INF, minIndex;
122         for (int v = 0; v < numVertices; v++) {
123             if (!mstSet[v] && key[v] < min) {
124                 min = key[v];
125                 minIndex = v;
126             }
127         }
128
129         // Add the picked vertex to the MST set
130         int u = minIndex;
131         mstSet[u] = true;
132
133         // Update key and parent index of the adjacent vertices
134         for (int v = 0; v < numVertices; v++) {
135             if (graph[u][v] && !mstSet[v] && graph[u][v] < key[v]) {
136                 parent[v] = u;
137                 key[v] = graph[u][v];
138             }
139         }
140     }
141 }

```

```
139     }
140
141     // Print the constructed MST
142     printf("Edges in the Minimum Spanning Tree (Prim's Algorithm):\n");
143     for (int i = 1; i < numVertices; i++) {
144         printf("%d -- %d == %d\n", parent[i], i, graph[i][parent[i]]);
145     }
146 }
147
148 // Main function to demonstrate MST algorithms
149 int main() {
150     // Example graph for Kruskal's Algorithm
151     int numVerticesKruskal = 4;
152     int numEdgesKruskal = 5;
153     Graph* graph = create_graph(numVerticesKruskal, numEdgesKruskal);
154
155     graph->edges[0] = (Edge){0, 1, 10};
156     graph->edges[1] = (Edge){0, 2, 6};
157     graph->edges[2] = (Edge){0, 3, 5};
158     graph->edges[3] = (Edge){1, 3, 15};
159     graph->edges[4] = (Edge){2, 3, 4};
160
161     kruskal_mst(graph);
162
163     // Example graph for Prim's Algorithm
164     int numVerticesPrim = 5;
165     int graphPrim[MAX_NODES][MAX_NODES] = {
166         {0, 2, 0, 6, 0},
167         {2, 0, 3, 8, 5},
168         {0, 3, 0, 0, 7},
169         {6, 8, 0, 0, 9},
170         {0, 5, 7, 9, 0}
171     };
172
173     prim_mst(graphPrim, numVerticesPrim);
174
175     free(graph);
176     return 0;
177 }
```

OUTPUT -

Edges in the Minimum Spanning Tree (Kruskal's Algorithm):

2 -- 3 == 4

0 -- 3 == 5

0 -- 1 == 10

Edges in the Minimum Spanning Tree (Prim's Algorithm):

0 -- 1 == 2

1 -- 2 == 3

0 -- 3 == 6

1 -- 4 == 5